

Trabalho de Formalização em SAT

Formalização e Verificação Experimental do Problema 2D-Sudoku (e da variante Sudoku-X)

Professor: Luis Henrique Bustamante
Equipe: Heitor Lacerda Santana de Sousa e Iago Fernando Barbosa Nunes Conserva
Repositório: <https://github.com/HeiLacerda/sudoku-sat-ufca.git>

1. Introdução

O Sudoku é um quebra-cabeça de preenchimento de grade amplamente conhecido: dada uma grade $N \times N$ dividida em N blocos de tamanho $\sqrt{N} \times \sqrt{N}$, o objetivo é preencher todas as células com números de 1 a N de modo que cada linha, cada coluna e cada bloco contenha cada número exatamente uma vez. Neste trabalho formalizamos o 2D-Sudoku e, como extensão natural, a variante conhecida como Sudoku-X, na qual as duas diagonais principais da grade também devem conter cada número exatamente uma vez.

O problema é interessante do ponto de vista computacional porque, apesar de sua formulação simples, pertence à classe de problemas de satisfação de restrições (CSP) e sua versão de decisão generalizada é NP-completa (Yato e Seta, 2003). Isso o torna um excelente estudo de caso para a modelagem em Lógica Proposicional: todas as restrições do jogo — de existência, de unicidade e de vizinhança — podem ser expressas de forma direta como fórmulas em Forma Normal Conjuntiva (FNC/CNF), permitindo que um SAT solver moderno decida a satisfatibilidade (e, quando satisfável, reconstrua uma solução) em frações de segundo mesmo para grades relativamente grandes.

Além do interesse prático, o Sudoku ilustra de forma clara conceitos centrais do curso: a codificação de restrições de cardinalidade ("exatamente um", "ao menos um", "no máximo um") como famílias de cláusulas, o crescimento polinomial (mas de alto grau) do número de cláusulas em função do tamanho da instância, e o efeito de se adicionar restrições extras (como a diagonal do Sudoku-X) sobre a satisfatibilidade de um conjunto de pistas previamente satisfável.

2. Formalização matemática

2.1 Definição das variáveis proposicionais

Seja N o tamanho da grade (com $n = \sqrt{N}$ o tamanho do bloco, exigindo-se que N seja um quadrado perfeito: 4, 9, 16, 25, ...). Definimos, para cada célula (i, j) com $1 \leq i, j \leq N$ e cada valor v com $1 \leq v \leq N$, a variável proposicional:

$$x[i,j,v] \Leftrightarrow \text{"a célula } (i, j) \text{ contém o valor } v"$$

A codificação linear utilizada para numerar as variáveis (exigida pelo formato DIMACS, que numera variáveis a partir de 1) é:

$$id(i, j, v) = (i - 1) \cdot N^2 + (j - 1) \cdot N + v$$

Essa codificação garante uma bijeção entre o conjunto de índices $\{1, \dots, N^3\}$ e o conjunto de tripla (i, j, v) , de modo que o número total de variáveis proposicionais é exatamente N^3 .

2.2 Famílias de cláusulas em CNF

As restrições do Sudoku são expressas por quatro famílias de cláusulas, aplicadas a quatro "grupos" de células (célula, linha, coluna, bloco), mais uma família opcional (diagonais). Cada grupo exige uma restrição de cardinalidade "exatamente um", que decomponemos em duas subfamílias: "ao menos um" (uma cláusula disjuntiva) e "no máximo um" (cláusulas binárias par a par, codificação direta/pairwise).

F1/F2 — Restrição de célula (define existência de valor)

Toda célula deve conter exatamente um valor:

F1 (ao menos um): $\bigvee x[i,j,v]$ para $v=1..N$, para cada (i,j)

F2 (no máximo um): $(\neg x[i,j,v] \vee \neg x[i,j,v'])$ para todo $v < v'$, para cada (i,j)

Justificativa: sem F1 uma célula poderia ficar vazia (nenhum valor atribuído); sem F2 uma célula poderia receber dois valores simultaneamente. A combinação das duas cláusulas é necessária e suficiente para forçar exatamente um valor por célula.

F3/F4 — Restrição de linha

F3 (linha contém v): $\bigvee x[i,j,v]$ para $j=1..N$, para cada linha i e cada valor v

F4 (v único na linha): $(\neg x[i,j,v] \vee \neg x[i,j',v])$ para todo $j < j'$, para cada linha i e cada valor v

F3 garante que cada valor de 1 a N aparece ao menos uma vez em cada linha; F4 impede que o mesmo valor apareça duas vezes na mesma linha. Juntas (e combinadas com F1/F2), forçam que cada linha seja uma permutação de $\{1, \dots, N\}$.

F5/F6 — Restrição de coluna

Análoga à restrição de linha, trocando o papel de i e j :

F5: $\bigvee x[i,j,v]$ para $i=1..N$, por coluna j e valor v . F6: $(\neg x[i,j,v] \vee \neg x[i',j,v])$ para todo $i < i'$.

F7/F8 — Restrição de bloco

Para cada bloco b (um dos $n \times n$ blocos de tamanho $n \times n$, com célula superior-esquerda (i_0, j_0)):

F7: $\bigvee x[i_0+d_i, j_0+d_j, v]$ para $0 \leq d_i, d_j < n$, por bloco e valor v

F8: $(\neg x[a,v] \vee \neg x[b,v])$ para todo par distinto (a,b) de células do mesmo bloco

Garante que cada bloco $n \times n$ também seja uma permutação de $\{1, \dots, N\}$, completando as três restrições clássicas do Sudoku (linha, coluna, bloco).

F9/F10 — Restrição de diagonal (variante Sudoku-X, opcional)

Para a variante com diagonais, adicionamos as mesmas restrições de "ao menos um" e "no máximo um" às células da diagonal principal $\{(k,k) : k=1..N\}$ e da diagonal secundária $\{(k, N+1-k) : k=1..N\}$:

F9: $\bigvee x[k,k,v]$ e $\bigvee x[k, N+1-k, v]$ para $k=1..N$, por valor v

F10: cláusulas pairwise de no-máximo-um análogas a F2/F4/F6/F8, restritas às células de cada diagonal

Essa família é a única que distingue o Sudoku-X do Sudoku clássico e, como mostrado na Seção 4, pode transformar um conjunto de pistas satisfatível para o Sudoku clássico em um conjunto insatisfatível quando exigida também a restrição diagonal.

F11 — Pistas (clues)

Para instâncias parcialmente preenchidas, cada pista fixando o valor v na célula (i,j) é simplesmente a cláusula unitária $(x[i,j,v])$.

2.3 Contagem de variáveis e cláusulas

O número de variáveis é sempre N^3 (independente do uso da restrição diagonal, já que nenhuma variável nova é criada — apenas cláusulas adicionais sobre variáveis já existentes).

Cada uma das quatro restrições clássicas (célula, linha, coluna, bloco) contribui com N^2 cláusulas de "ao menos um" e $N^2 \cdot C(N,2)$ cláusulas de "no máximo um" (onde $C(N,2) = N(N-1)/2$ é o número de pares de valores). Assim, o número total de cláusulas sem diagonal é:

$$M(N) = 4N^2 + 4N^2 \cdot C(N,2) = 4N^2 + 2N^3(N-1) = 2N^4 - 2N^3 + 4N^2$$

Com a restrição diagonal (duas diagonais, cada uma com N células), somam-se $2N$ cláusulas de "ao menos um" e $N^2(N-1)$ cláusulas de "no máximo um":

$$M_{diag}(N) = M(N) + 2N + N^2(N-1) = 2N^4 - N^3 + 3N^2 + 2N$$

A tabela a seguir mostra os valores para os tamanhos de instância utilizados nos experimentos (Seção 4). Os valores foram conferidos automaticamente pelo gerador (Seção 3.3) e coincidem exatamente com os cabeçalhos DIMACS produzidos.

N	n=raiz(N)	Variaveis (N^3)	Clausulas (sem diagonal)	Clausulas (com diagonal)
4	2	64	448	504
9	3	729	11988	12654
16	4	4096	123904	127776
25	5	15625	752500	767550

Observa-se o crescimento de $M(N)$ como $O(N^4)$: dobrar N multiplica o número de cláusulas por um fator próximo de 16, o que já se manifesta claramente ao passar de $N=9$ (≈ 12 mil cláusulas) para $N=25$ (≈ 750 mil cláusulas).

3. Implementação

3.1 Linguagem e interface

O gerador foi implementado em Python 3 (generator/gerador.py), executável via linha de comando conforme exigido no enunciado:

```
python3 gerador.py N [--clues K] [--seed S] [--diagonal] [--force-unsat] [--load-grid ARQ] >
instancia.cnf
```

O parâmetro N define o tamanho da grade; `--clues` define quantas células vêm pré-preenchidas; `--diagonal` ativa as famílias F9/F10; `--force-unsat` insere deliberadamente duas pistas conflitantes na

mesma linha, produzindo uma instância UNSAT para fins de verificação; --load-grid permite fornecer um grid-solução externo (usado, por exemplo, para garantir pistas consistentes com a restrição diagonal — ver Seção 4).

3.2 Geração do grid-solução base

Para gerar pistas consistentes, o código primeiro constrói um grid-solução válido: parte de uma grade cíclica determinística (linha i , coluna $j \mapsto ((i \cdot n + Li/nJ + j) \bmod N) + 1$), depois embaralha bandas de linhas e de colunas preservando a estrutura de blocos, e por fim reindexa os símbolos por uma permutação aleatória. Isso garante uma solução válida (linhas, colunas e blocos corretos) sem exigir uma busca custosa. As pistas são então extraídas amostrando aleatoriamente K células desse grid.

Um detalhe relevante (discutido na Seção 4): esse procedimento não garante que o grid gerado satisfaça também a restrição diagonal do Sudoku-X — a permutação de bandas de linha/coluna preserva blocos, mas não as diagonais. Por isso, para gerar instâncias diagonais consistentes com uma solução real, o código pode alternativamente carregar (--load-grid) um grid obtido resolvendo-se a instância diagonal em branco com o próprio SAT solver — uma forma direta de "bootstrap" que usamos nos experimentos.

3.3 Verificação de sanidade

Antes de emitir o arquivo, o gerador recalcula o número de variáveis realmente usadas nas cláusulas e verifica que ele bate com N^3 , além de emitir o cabeçalho DIMACS ($p \text{ cnf } N_vars \text{ } N_clauses$) com a contagem exata de cláusulas geradas — evitando o erro citado no enunciado, em que um cabeçalho incorreto faz o solver rejeitar o arquivo.

```
assert max(used_vars) <= n_vars
assert n_vars == n_vars_expected # bate com  $N^3$ 
```

3.4 Reconstrução da solução

O script generator/reconstruct.py lê a saída padrão do CaDiCaL (linhas "s SATISFIABLE"/"v ..."), filtra os literais positivos e, para cada célula (i,j) , procura o único v tal que $x[i,j,v]$ é verdadeiro, produzindo a grade final legível.

3.5 Repositório

O código está organizado em um repositório Git com histórico de commits incrementais (estrutura inicial → reconstrução → script de experimentos → resultados → README) e um README com instruções de uso. O link do repositório (a ser publicado pela equipe no GitHub/GitLab) deve ser inserido na capa deste relatório antes da entrega.

4. Experimentação

Os experimentos foram executados com o SAT solver CaDiCaL (versão 3.0.0, compilada a partir do código-fonte oficial) em 10 instâncias de tamanhos e configurações crescentes, cobrindo grades de $N=4$ a $N=25$, instâncias em branco, parcialmente preenchidas, com e sem restrição diagonal, e um caso deliberadamente UNSAT. O tempo de execução foi medido por relógio de parede (equivalente ao uso de time cadical instancia.cnf sugerido no enunciado).

Instancia	N	Diag.	Pistas	Variaveis	Clausulas	Resultado	Tempo (s)
sudoku_4_blank	4	nao	0	64	448	SATISFIABLE	.003671650

sudoku_4_c6	4	nao	6	64	454	SATISFIABLE	.003582131
sudoku_9_blank	9	nao	0	729	11988	SATISFIABLE	.030513930
sudoku_9_c17	9	nao	17	729	12005	SATISFIABLE	.007392123
sudoku_9_c30	9	nao	30	729	12018	SATISFIABLE	.006714301
sudoku_9_unsat	9	nao	30	729	12018	UNSATISFIABLE	.006228638
sudokux_9_c30_unsat	9	sim	30	729	12684	UNSATISFIABLE	.006643051
sudokux_9_c30_sat	9	sim	30	729	12684	SATISFIABLE	.006596494
sudoku_16_c50	16	nao	50	4096	123954	SATISFIABLE	.034932899
sudoku_25_c100	25	nao	100	15625	752600	SATISFIABLE	.953067717

Reconstrução de uma solução (exemplo, instância sudoku_9_c17, N=9 com 17 pistas):

```

8 7 3 6 2 1 5 9 4
6 9 2 8 5 4 1 7 3
5 1 4 7 3 9 8 2 6
4 6 7 9 8 5 3 1 2
3 5 9 1 7 2 4 6 8
1 2 8 3 4 6 9 5 7
9 4 5 2 6 3 7 8 1
2 8 1 4 9 7 6 3 5
7 3 6 5 1 8 2 4 9

```

Um resultado experimental notável envolve o par sudokux_9_c30_unsat / sudokux_9_c30_sat: usamos o mesmo número de pistas (30) e a mesma semente para escolher quais células revelar, mas extraídas de dois grids-solução diferentes. No primeiro caso, as pistas foram extraídas do grid-solução "padrão" (válido para linha/coluna/bloco, mas não necessariamente para as diagonais) — ao adicionar a restrição diagonal (F9/F10), a instância torna-se UNSAT, mesmo sem qualquer conflito direto entre pistas na mesma diagonal. No segundo caso, as pistas foram extraídas de um grid-solução obtido resolvendo-se previamente a instância diagonal em branco (bootstrap via SAT solver) — dessa vez o resultado é SAT. Isso demonstra experimentalmente que a satisfatibilidade de um conjunto de pistas depende criticamente de qual variante do problema (Sudoku clássico vs. Sudoku-X) está sendo resolvida, mesmo quando as pistas em si não violam nenhuma restrição de forma direta e óbvia.

5. Análise e conclusão

Os experimentos revelam três comportamentos principais do solver e da formalização:

- Escalabilidade: mesmo com o crescimento $O(N^4)$ do número de cláusulas (Seção 2.3), o CaDiCaL resolve todas as instâncias testadas — incluindo N=25, com mais de 750 mil cláusulas — em menos de 1,1 segundo. Isso é consistente com a literatura (Sudoku 9×9 é resolvido em microssegundos por solvers modernos; ver material de referência da disciplina) e evidencia que, apesar de o problema ser NP-completo em geral, instâncias práticas com estrutura combinatória forte (como o Sudoku) são tipicamente fáceis para CDCL solvers modernos.
- Efeito das pistas: instâncias em branco (sem pistas) e com poucas pistas (17, próximo do mínimo teórico conhecido de um Sudoku 9×9 com solução única) foram resolvidas praticamente no mesmo tempo que instâncias com mais pistas (30) — o gargalo não é o número de pistas, mas o tamanho da grade N.
- Diagonal como restrição estritamente mais forte: como discutido na Seção 4, a restrição diagonal (Sudoku-X) pode tornar UNSAT um conjunto de pistas que seria SAT no Sudoku clássico. Isso ilustra formalmente que o conjunto de modelos do Sudoku-X é um subconjunto estrito do

conjunto de modelos do Sudoku clássico (toda solução de Sudoku-X é solução de Sudoku, mas a recíproca não vale), e que a consequência semântica "pistas = solução" não é preservada ao se adicionar restrições.

Quanto à eficiência da formalização: a codificação pairwise usada para as cláusulas de "no máximo um" é simples de justificar corretamente (Seção 2.2), mas não é assintoticamente ótima — ela produz $O(N^2)$ cláusulas por grupo, enquanto codificações alternativas (por exemplo, codificação binária/logarítmica com $O(\log N)$ variáveis auxiliares por grupo, ou a codificação sequencial de Sinz) produzem menos cláusulas às custas de variáveis auxiliares adicionais. Como o CaDiCaL já resolve as instâncias testadas em frações de segundo mesmo com a codificação pairwise, não observamos necessidade prática de uma codificação mais compacta nesta escala ($N \leq 25$); a comparação experimental entre codificações fica registrada aqui como direção natural de extensão do trabalho (nível médio dos requisitos adicionais do enunciado), assim como a aplicação da transformação de Tseitin a sub-fórmulas mais complexas (por exemplo, ao codificar a soma de gaiolas do Killer Sudoku) e a comparação com um segundo solver (Kissat).

Em conclusão, a formalização proposta é correta (produz somente cláusulas necessárias e, em conjunto, suficientes para capturar exatamente as regras do Sudoku e, opcionalmente, do Sudoku-X), foi verificada automaticamente por checagem de sanidade no gerador, e mostrou-se computacionalmente tratável mesmo para grades bem maiores do que as tipicamente jogadas por humanos ($N=25$), confirmando a robustez de SAT solvers CDCL modernos para este tipo de problema estruturado.

Referências

- [1] BUSTAMANTE, L. H. Trabalho de Formalização em SAT — Lógica para Ciência da Computação, UFCA, 2026.1.
- [2] BUSTAMANTE, L. H. Problemas para Formalização em Lógica Proposicional e SAT, UFCA, 2026.1.
- [3] BIERE, A.; HEULE, M.; VAN MAAREN, H.; WALSH, T. Handbook of Satisfiability. IOS Press, 2ª ed., 2021.
- [4] AVIGAD, J. Mathematical Logic and Computation. Cambridge University Press, 2022.
- [5] BIERE, A. CaDiCaL SAT Solver. Disponível em: <https://github.com/arminbiere/cadical>.
- [6] YATO, T.; SETA, T. Complexity and Completeness of Finding Another Solution and Its Application to Puzzles. IEICE Transactions on Fundamentals, 2003 (prova de NP-completude do Sudoku generalizado).